

Projet Java L3 Miage

« Vous êtes le héros »

1. Dates importantes :

- Rendu du projet : le mardi 02/07/2024 à 20h.
- Les séances de suivi : à fixer ultérieurement

2. Description

Ce projet consiste à produire un jeu du type “vous êtes le héros”. Ce style de jeu très populaire s’adapte à différents types de récit : aventure, romance, horreur; et permet de plonger le.la joueur.euse dans une histoire dont l’issue sera déterminée par ses choix et possiblement des événements aléatoires (e.g., un choix à pile ou face). Les choix réalisés seront déterminants car ils impacteront l’issue du récit à un moment plus ou moins proche des décisions prises, assurant au joueur (à la joueuse) une fin heureuse ou terrible.

Les différents choix possibles mèneront donc à différents parcours de jeu. Tous ces parcours mènent à une représentation sous forme d’un graphe orienté acyclic (acyclic pour éviter une partie infinie). Le graphe est composé de nœuds (chaque nœud représentant une situation) et d’arcs (chaque arc représentant une transition entre deux nœuds). Il existera différents types de nœud, des nœuds de *décision* pour lesquels le.la joueur.euse est amené.e à faire un choix, des nœuds *chance* où un événement aléatoire se produit, et des nœuds *terminaux* pour la fin du jeu (e.g., le.la joueur.euse termine vainqueur.euse d’une lutte épique couvert.e de lauriers).

Pour immerger le.la joueur.euse dans un univers riche et immersif, on travaillera également, une hiérarchie sur les personnages possibles du jeu. Vous serez libre de choisir cet univers mais vous devrez respecter certaines contraintes. En effet, l’ensemble de vos classes devront présenter des relations d’héritage et de composition, et vous devrez utiliser des interfaces, des classes abstraites, et des énumérations.

Par exemple (vous devez donc proposer quelque chose d’autre), on pourrait imaginer un univers d’aventure fantastique dans lequel trois peuples (les orcs, les humains, et les elfes) se battent dans une guerre sans merci. Dans cet univers, le.la joueur.euse pourra incarner différents personnages (classe Personnage). Au fur et à mesure de l’histoire le.la joueur.euse fera des choix qui le.la mèneront à devenir soit un.e soldat.e soit un.e mage (classe Mage). La classe Soldat est une classe abstraite dont dérive les classes Lancier ou Archer. Il y a ici des relations d’héritage entre les classes Personnage, Mage, Soldat, Lancier, et Archer. Les classes Archer et Mage implémentent une interface commune *AttaqueADistance*. Pour les énumérations, nous pouvons en utiliser une pour représenter les différents peuples (orcs, humains, elfes), ou encore une pour représenter les différents grades d’un soldat (caporal, sergent, capitaine, commandant). Enfin, pour la composition, nous pouvons utiliser le fait que chaque personnage dispose d’une âme et créer une autre hiérarchie de classes à partir d’une classe mère *Arme*.

Ce travail est à réaliser **en binôme**. En plus de votre code, vous devrez rendre un *rapport* contenant :

- Une présentation de la structure de votre code, et une justification des choix concernant vos hiérarchies de classes.
- Une présentation de vos extensions éventuelles.
- Une discussion sur ce qui a été le plus dur à implémenter pour vous et sur le niveau de difficulté du projet.

Votre projet devra également contenir un fichier *ReadMe* présentant succinctement votre projet et détaillant les instructions à réaliser pour le compiler, l'exécuter, puis l'utiliser.

3. Planning de travail

Travail sur le package representation.

- Créer un *package representation* qui contiendra les classes représentant le graphe des états possibles de votre jeu.
 - Dans le package *representation*, créer les *classes* *Node*, *InnerNode*, *ChanceNode*, *DecisionNode*, et *TerminalNode*.
 - Un *Node* représente un nœud du graphe de l'histoire. Il présente une situation au joueur (à la joueuse) à partir d'un champ `String description` et d'une méthode qui affiche cette description `public void display()`. Chaque instance de la classe *Node* aura un attribut `int id`, correspondant à un identifiant unique attribué à l'instance. Chaque *Node* devra également être combiné par composition avec les classes de votre package *univers* (voir prochain paragraphe). Un *Node* possède une méthode `public Node chooseNext()` permettant de désigner le prochain nœud à considérer.
 - Un *TerminalNode* est un nœud terminal du graphe (et donc la fin de l'histoire). La méthode `chooseNext()` devra donc renvoyer le nœud lui-même.
 - Un *InnerNode* est un nœud non terminal du graphe. Pour simplifier nous supposons dans un premier temps qu'un *InnerNode* peut mener à quatre autres nœuds renseignés en attributs.
 - Un *DecisionNode* est un nœud où le joueur doit prendre une décision. Ainsi la méthode sélectionnant le prochain nœud devra interagir avec l'utilisateur, e.g., via la console. Vous pouvez regarder la documentation de la classe *Scanner* ici.
 - Un *ChanceNode* est un nœud où un événement mêlant une part d'aléatoire intervient. Ainsi la méthode sélectionnant le prochain nœud devra simuler une loi aléatoire. Vous pouvez regarder la documentation de la classe *Random* ici.
- Quelles relations d'héritage instaurerez-vous entre ces classes ; avec quelles redéfinitions ?

Travail sur le package univers.

- Créer un *package univers* qui contiendra les classes représentant l'univers dans lequel va évoluer le joueur.
- Créer les premières classes de votre univers, notamment avec une classe "personnage de base".

Sur les deux packages.

- Travailler sur la *visibilité* de vos attributs et méthodes.
- Réaliser des *getters*, des *setters*, et des *constructeurs*. Faites attention au lieu entre héritage et constructeurs.
- Redéfinir les méthodes `toString` et `equals` de la classe `Object` dans vos différentes classes (documentation ici).
- Écrire une méthode `main` pour tester votre programme. Vous devriez pouvoir simuler une toute petite histoire.
- Déterminer si certaines de vos classes devraient être *abstraites*.
- Continuer à travailler sur le package `univers` pour rajouter des énumérations, des classes, abstraites, des interfaces, et de la composition. Vous devez obtenir au moins une petite dizaine de classes en comptant tout.
- Voici un point un peu plus avancé ! On souhaite maintenant rajouter des fonctionnalités à nos nœuds pour qu'ils puissent, avant l'affichage de la description de la situation, afficher une image ou jouer un son. Pour ce faire, au lieu de complexifier notre hiérarchie, nous allons utiliser le design pattern *Decorator*.
- Créer une interface `Event` qui sera implémentée par la classe `Node`. Elle demandera l'implémentation des méthodes :
 - **public void display();**
 - **public Displayable chooseNext();**
- Créer un décorateur abstrait implémentant `Event` et deux décorateurs concrets (`ImageNode` et `SoundNode`) selon le design pattern *Decorator*.
- Pour jouer un son avec Java, chercher des informations sur les classe du package `javax.sound.sampled`, **puis** n'hésitez pas à demander des conseils à votre chargé.e de TD/TP ou au responsable d'UE.
- Pour montrer une image avec Java, chercher des informations sur les classes des packages `java.awt` et `java.swing`, **puis** n'hésitez pas à demander des conseils à votre chargé.e de TD/TP ou au responsable d'UE.
- Différencier désormais une classe `Game` qui modélise la partie jouer par le joueur de la classe `Main` qui configurera la partie si besoin et lancera le jeu (avec une instance de la classe `Game`).
- Placer votre projet sur *github* afin de pouvoir *versionner* votre code pour la suite du projet.
- Commencer la *documentation Javadoc* de votre projet. Celle-ci devra être relativement complète au moment du rendu.
- Choisir deux classes de votre choix et créer deux classes de *tests* `JUnit` pour ces classes afin de réaliser des *tests unitaires*. Toutes les méthodes ne devront pas être testées mais vous devait écrire une variété de tests illustrant les possibilités de l'outil `JUnit`.
- Rajouter les *annotations* nécessaires à votre projet telles que les `@Override`.
- Dans la classe `InnerNode`, faites en sorte qu'il puisse y avoir un nombre non défini (et pas forcément quatre comme précédemment défini) de prochains nœuds possibles en utilisant une collection de nœuds comme attribut.
- Rajouter des collections ou des maps dans votre package `univers`.
- Grâce à la sérialisation proposer un système de sauvegarde de partie. Lors d'une partie le joueur aura la possibilité de sauvegarder la partie afin de reprendre plus tard.
- Créer un menu interactif afin d'interagir dans la console avec l'utilisateur.ice de votre programme dans la classe `Main`. Le joueur peut alors quitter le programme, lancer une nouvelle partie, ou reprendre une sauvegarde. Les sauvegardes devront être présentées de la plus récente

à la plus ancienne.

- Gérer les différentes *exceptions* possibles de votre programme. Il faudra notamment gérer les exceptions générées par les flux d'entrées-sorties et les exceptions générés par l'utilisation d'arguments inappropriées (par exemple, un nombre de munitions négatif pour un archer).

Pour la suite : Si vous voulez aller plus loin (fortement recommandé de le faire) :

- Réaliser une interface graphique digne des plus grands jeux vidéo.
- Gérer le fait qu'il puisse y avoir plusieurs joueurs.euses, chacun gérant plusieurs parties disjointes.

4. Binômes :

1	Asma Ben-Zine	Marie Probert
2	Maxance Villame	Ferdinand Martin-Lavigne
3	Vladislav Yaromiy	Iana Gerliakh
4	Raiydane Bundhoo	Thi tho Pham
5	Dimitry SALEH	Jefferson GARIBALDI
6	Dan CHEMLA	Victor Mendez
7	Sofia ABAT	Fayçal BENAÏSSA
8	Lounes BOUDINAR	Melina MAOUCHE
9	Ouasti Redouane	Barry saidou
10	Simon Regenwetter	Aghiles AZIZI
11	Lahcene Boussekine	Neyla Illouli